

Orbit Academy — Security Notes & Audit Log

v5.5

Contents

Orbit Academy — Security Notes & Audit Log	1
v5.5 — audit result: PASS	1
v5.4 — audit result: PASS	2
v5.3 — audit result: PASS	2
v5.2 — audit result: PASS	2
v5.1 — audit result: PASS	3
v5.0 — audit result: PASS	4
v4.1 — audit result: PASS	5
v4.0 — audit result: PASS	6
v3.1 — audit result: PASS	6
v3.0 — audit result: PASS	6
v2.5 — audit result: PASS	7
v2.4 — audit result: PASS	7
v2.3 — audit result: PASS (re-run)	7
v2.2 — audit result: PASS (re-run)	8
v2.1 — audit result: PASS (re-run)	8
v2.0 — audit result: PASS (re-run)	8
v1.3 — audit result: PASS (re-run)	9
v1.2 — audit result: PASS (re-run)	9
v1.1 — audit result: PASS (re-run)	9
Threat model	9
Audit checklist (run every version)	10
v1.0 — audit result: PASS	10

Orbit Academy — Security Notes & Audit Log

Each release passes a security audit before it is pushed. This backend has a **real auth surface** (accounts, password hashing, sessions, roles, progress writes), so the audit is more involved than a static toy.

v5.5 — audit result: **PASS**

Client-side content release (Module 7 worksheet/sim text revision from an external review; one worksheet-engine cosmetic field). No new endpoints, inputs, file reads or auth changes; the `server.js` diff is the version header only. All new worksheet strings are

static data-file content rendered through the existing engine pipeline; the new tutorial table is static HTML with no user data (the `esc()` discipline applies to user data, which this release does not touch). Full checklist re-run 2026-09-21 via `test/run.sh`: path traversal, unauthenticated access, privilege escalation, session forgery, prerequisite bypass, input validation, DoS body-size, XSS, secrets in repo (`academy_data.json` gitignored — verified). Static + runtime network monitoring unchanged: zero outbound calls (the revised text adds no I/O of any kind).

v5.4 — audit result: PASS

Client-side release (module 6 scenario + view/clock fixes, module 8 legend and exposure, wording, tests, docs). No new endpoints, inputs, file reads or auth changes; the `server.js` diff is the version header only. Full checklist re-run 2026-07-31: path traversal, unauthenticated access, privilege escalation, session forgery, prerequisite bypass, input validation, DoS body-size, XSS, secrets in repo (`academy_data.json` gitignored — verified). Static + runtime network monitoring unchanged: zero outbound calls. The new audio and halo code add no I/O of any kind.

v5.3 — audit result: PASS

No change to the attack surface: v5.3 is a client-side release (simulator rendering, worksheet content, layout, tests). No new endpoints, no new inputs, no new file reads, no auth changes. `server.js` diff for this release is the version header only. Full checklist re-run 2026-07-30: path traversal, unauthenticated access, privilege escalation, session forgery, prerequisite bypass, input validation, DoS body-size, XSS (all new worksheet strings pass through the same data-file pipeline and `esc()` discipline), secrets in repo (`academy_data.json` gitignored; verified). Static + runtime network monitoring unchanged: zero outbound calls.

v5.2 — audit result: PASS

New surface this cycle: three instructor-only endpoints (`/api/user/delete`, `/api/user/reset`, `/api/analytics`), one unauthenticated endpoint (`/api/setup`), and wrong-answer recording on `/api/task`. Security suite grew **30 → 48 checks**.

Decision recorded: no default account or password ships with the course. A fixed instructor credential in the repository would be identical on every installation worldwide, would persist in git history permanently, and is the classic hard-coded-credentials weakness (CWE-798) — a predictable finding in any accreditation review, and unfixable after the fact. The problem it was meant to solve (a stray test account claiming the instructor role, stranding the real instructor as a student) is solved instead by `GET /api/setup`, which returns a single boolean — whether any account exists — so a fresh server can state that the next account created becomes the instructor. Disclosure is limited to “is this instance configured yet”, it is only ever true once, and on a closed training LAN it is not sensitive. Asserted to return exactly the one key.

Role changes remain off the API. An endpoint granting instructor rights would be the most valuable privilege-escalation target here. Promotion is offline via `tools/set-role.js`, which grants nothing to anyone who could not already read the data file.

Destructive endpoints. Verified: student and anonymous callers are refused (403/401) on delete, reset and analytics; an instructor cannot delete their own account; the **last** instructor cannot be deleted; unknown accounts return 404; deleting a user purges their sessions so a live cookie cannot outlive the account. With two instructors present the last-instructor guard correctly relaxes — covered by `test/multi-instructor.test.js`.

Progress integrity. Wrong answers are now reported, which introduced a way to send `done:false` for an already-completed task. `done` is therefore monotonic server-side: a miss (or a replayed request) can never un-complete a module a student has passed. Asserted directly.

Durability. Debounced writes could be lost if the process was killed inside the 100 ms window — and closing the launcher window is a kill. Now flushed on SIGINT/SIGTERM/exit.

Data-handling defect found and fixed in the test harness, not the product: `two-run-modes.test.js` used `ACADEMY_DATA` where the server reads `ORBIT_DATA`, so a test server silently fell back to the live `academy_data.json` and registered accounts into real student records. Nothing was lost, but the exposure was real. The test now keeps its store in the OS temp directory and asserts the live file was never written; `tools/set-role.js` had the same wrong variable name.

Distribution. `tools/make-bundle.sh` refuses to build if `academy_data.json` (password hashes, student progress) would be included, if a required asset is missing, or if any CDN reference survives in `public/*.html`, and emits a SHA-256 for publication.

Re-verified unchanged: path traversal, unauthenticated access, privilege escalation, session forgery, prerequisite bypass, input validation, body-size limits, XSS via worksheet/roster data, secrets hygiene. `academy_data.json` confirmed gitignored.

Suites: functional 34 ✓ · security 48 ✓ · DoS ✓ · air-gap ✓ · vendor integrity 5/5 ✓ · both run modes ✓ · file:// robustness 13/13 ✓ · multi-instructor 17/17 ✓ · Module 8 cockpit 17/17 ✓ · Module 6 physics 5/5 ✓.

v5.1 — audit result: PASS

Scope of change since v5.0: all vendored assets committed to the repository; `fetch-vendor.sh --check` turned into a hash verifier and gated in `test/run.sh`; two `file://` robustness fixes in client code; install documentation rewritten. No change to `server.js` logic beyond its version string, so the `auth/session/roster` surface is unchanged from the v5.0 audit and its 30 security checks still pass.

Supply chain — the substantive new consideration. Committing third-party binaries means the repo itself is now the trust boundary, so:

- `public/vendor/NOTICE.md` records origin, license and SHA-384 for all five files. `three.js` is MIT (notice intact in the file's own header); the two photographic planet maps ship in the `three.js` examples tree and derive from NASA imagery; the two schematic maps are ours, reproducible from `tools/make-textures.py` with a fixed seed.
- The `three.js` digest **equals the published r128 SRI hash** — the same value the old CDN integrity= attribute pinned. Vendoring provably did not alter the library.
- `bash tools/fetch-vendor.sh --check` recomputes all five hashes and **fails the suite** on mismatch or on a missing required file. Verified by appending a single

byte to moon_1024.jpg (detected) and by deleting three.min.js (detected, exit 1). Optional photo maps absent are reported, not failed.

- Net effect on attack surface: **reduced**. A default install now issues no outbound request at all, so there is no CDN, DNS or TLS dependency left to attack or to be blocked by a proxy.

Client robustness (both were denial-of-function bugs on file://, not security holes):

- public/index.html — fetch('/api/me') *rejects* rather than resolving !ok when there is no origin, so the hub hung blank with no diagnostic. Now caught, with an explanation and a link to the no-server entry point. No change to what the server will accept.
- public/tut8.html — getImageData on a canvas holding a file:// image raises SecurityError (tainted canvas). Now guarded and cached as a known failure, with an on-screen label, so the render loop cannot be killed by it.

Re-verified this cycle: path traversal, unauthenticated access, privilege escalation, session forgery, prerequisite bypass, input validation, request-body limits, XSS via worksheet/roster data, and secrets hygiene — all pass unchanged. academy_data.json confirmed gitignored; the new public/vendor/ entries were reviewed to be sure the ignore-rule edit exposed nothing else.

Suites: functional 34 □ · security 30 □ · DoS □ · air-gap 16 checks / 0 outbound calls □ · vendor integrity 5/5 □ · Module 8 cockpit 17/17 □ · Module 6 physics 5/5 □.

v5.0 — audit result: PASS

The v5.0 change set is large but almost entirely **client-side content and assets**; server.js changes only its version-header comment — zero changes to auth, sessions, roles, routing, parsing, body-size limits, path handling, or the workbook editor. Audit performed 2026-07-28.

Scripted suites (bash test/run.sh, fresh isolated servers): functional **34** □, security **30** □ (path traversal ×4, unauth access ×4, first-user-instructor, role self-assignment, priv-esc, session forgery, prereq bypass ×2, score clamping, password/email validation, malformed JSON, workbook-editor authz ×4 + id validation ×2, unknown-worksheet route, post-logout invalidation), DoS 300 KB body □ — **ALL SUITES PASSED**, plus the new air-gap suite (below).

New in this release — the client is verified to make NO outbound network calls.

This matters because the course is deployed on standalone / air-gapped systems. - three.js now loads from public/vendor/three.min.js in all eight simulators (was a pinned CDN tag). tools/fetch-vendor.sh fetches it once and **verifies the published SHA-384**, refusing to install a library whose hash does not match — the same integrity guarantee the SRI tag gave, now enforced at install time instead of per page load. - Planet textures resolve through public/textures.js with ALLOW_CDN = false by default; the final fallback is a **schematic map committed to the repo**, so no asset is ever fetched. - **Removed the optional Anthropic tutor** from Module 1 (a user-supplied API key POSTed the orbit to api.anthropic.com). No key ever shipped and it was inert by default, but an air-gapped deployment should not contain a field capable of carrying a credential off-box. The built-in local explainer covers the same ground. - Enforcement:

test/no-external-calls.test.js, now part of test/run.sh. Two independent passes — a static scan for fetchable constructs (src=, <link href>, url(), fetch(), XHR.open, WebSocket, EventSource, sendBeacon, importScripts, .src=) and a runtime pass that executes all eight simulators against instrumented network primitives. Result: **0 outbound calls from every module**; 99 <a href> further-reading links remain and are inert. The test itself was validated by injecting a CDN script tag, a fetch() and an Image().src and confirming both passes caught all three.

New client code reviewed. textures.js builds its DOM banner with createElement/textContent (the one innerHTML use is a static template string, no user data); tools/make-textures.py is a build-time generator that touches no user data; the new Module 1 controls card is static markup with a localStorage seen-flag; test/* are developer tools, never served. No eval, no document.write, no new storage or network access anywhere.

Physics / functionality re-verified on the release candidate. Module 8 headless cockpit harness (test/tut8-cockpit.verify.js): both reference missions fly to completion — GEO rendezvous on station T+1.71 d on 3,896/6,500 m/s, lunar capture held with three burns T+5.44 d on 3,728/9,500 m/s, **17/17 checks**. That harness was itself extended this release to execute the page's local <script src> files, which immediately caught a real breakage (tut8 using 0A_TEX before textures.js was loaded).

Module 6 fan suite — re-run in full, all five scenarios PASS. test/tut6-physics.verify.js is documented as taking ~40 s *per mode* and is meant to be run per mode (node test/tut6-physics.verify.js public/tut6.html scatter|llknife|tadpole|dro|freeret). Each was executed against the shipped tut6.html functions and arrays extracted verbatim:

mode	result	key figures
scatter	PASS	yellow flung to 1.54 M-km; the other six bound at 1.2-12.8 R_M
llknife	PASS	3 moonward, 2 earthward, 2 hovering — the knife edge intact
tadpole	PASS	red breaks out at 180°; the rest bounded 1.1-22.4°
dro	PASS	every DRO bound 90 d; far prograde stripped; 30k prograde impacts d38.6
freeret	PASS	pink returns d18.0, green d7.8, cyan d8.3, pale d13.3 — 4 of 7 home

academy_data.json confirmed gitignored. Cleared to ship v5.0.

v4.1 — audit result: PASS

The v4.1 change set is the owner flight-test round on the v4.0 fan lab plus the star-field upgrade — **entirely client-side** (tut1-8 star fields, tut6 warp/halos/Hill-ring/banner/zoom, tut7.js camera stars, worksheet 3/6 text, README/CHANGELOG); the ONLY server.js change is the version-header comment. Audit performed 2026-07-23: bash test/run.sh → functional + security (30) + DoS — **ALL SUITES PASSED** on fresh isolated servers. New client code review: capture-halo sprites, Hill ring, and banner-clearing touch no user data and no innerHTML with non-course strings; star generators use Math.random() only for cosmetics; no eval/document.write; no new network or storage access. Physics re-verified per MAINTENANCE §6.5 on the release candidate: all five deterministic fan

suites PASS (test/tut6-physics.verify.js, free-return window extended to 20 d showing pink's day-18 return), minimoon set verified over 300 d at a non-zero release time. academy_data.json confirmed gitignored. Cleared to ship v4.1.

v4.0 — audit result: PASS

The v4.0 change set is **entirely client-side content** (tut3/tut6 physics + UI, worksheet 3/6 data, instructor guide + regenerated guide.html, module3/6 decks, README/CHANGELOG); the ONLY server.js change is the version-header comment — zero changes to auth, sessions, roles, routing, parsing, or file access. Audit performed 2026-07-23 on this basis plus the full scripted suites: bash test/run.sh → functional 28+ □, security 30 □ (path traversal, unauth access, priv-esc, session forgery, prereq bypass, input validation, workbook-editor authz + id validation), DoS 300 KB body □ — **ALL SUITES PASSED**. New client code review: worksheet strings render through the existing engine (quiz data is trusted course content, no user data in innerHTML paths); tut3 sun HUD and tut6 fan messages are static template strings (no user input); no eval/document.write; the one new DOM node (sun HUD) is pointer-events:none. academy_data.json confirmed gitignored (git check-ignore). Physics verification per MAINTENANCE §6.5: all six tut6 fan scenarios + tut3 J2 rates re-verified by executing the shipped code verbatim on the release candidate. Cleared to ship v4.0.

v3.1 — audit result: PASS

The v3.1 change set is the post-3.0 flight-test series (two rounds; the second via a prior CLI session, folded in and verified here): Module 8 cockpit upgrades (autopilot, live guidance, countdowns, local-frame thrust, gates, collision, budgets), Module 1/3/5/6 content and physics fixes, onboarding docs, and distributable-bundle tooling. **All client-side** except two console.error strings in server.js (a friendlier port-in-use hint) — zero changes to auth, sessions, roles, gating, body-size limits, path handling, or the workbook editor. New client surface reviewed: the autopilot is pure client JS driving the existing thrust handlers (nothing network-facing, no new storage); no new external dependencies or CDN tags; distributable zips are gitignored build artifacts. Reference solutions re-verified against the flight model (GEO + Moon both captured through the new pre-coast schedule). Full test suite re-run this date: **functional + security + DoS — all passing**. academy_data.json confirmed gitignored. Cleared to ship v3.1.

v3.0 — audit result: PASS

The v3.0 change set is overwhelmingly **client content** (worksheet data, tut HTML/JS, docs): the QA-walkthrough fixes, the Module 8 lunar-mission enablement, instructor materials (decks, docs, launchers), and Module 1's surface-clipped re-entry arcs. server.js changed in three strings — the version header, the t1 course title, and a friendlier port-in-use message — with **zero changes** to auth, sessions, roles, gating, body-size limits, path handling, or the workbook editor. The new launcher scripts (start-academy.bat/.command) only check for node, open a localhost URL, and run node server.js — no privileged operations, nothing downloaded. New client surface reviewed: tut8 now loads Three.js from cdnjs **SRI-pinned** (same tag as tut1); the lunar unlock is a cosmetic localStorage flag (m7_geo_done) — it gates only a planner button, and module completion remains server-enforced, so “unlocking” it by hand grants

nothing. Quiz-option shuffling is render-side only; answer keys never leave the data files (unchanged posture). Full test suite re-run this date: **functional + security + DoS body-size — 30+ checks, all passing** (test/run.sh). academy_data.json confirmed gitignored (verified with git check-ignore). Cleared to ship v3.0.

Process note: v2.6 shipped without a logged entry here (caught by the v3.0 QA walk-through — finding #64). The v2.6 change set was client/docs-only plus the COURSE re-ordering; this v3.0 audit covers the cumulative state. The release checklist gate stands: no push without a logged PASS.

v2.5 — audit result: PASS

The v2.5 change set is **client + docs only**: the Module 6 simulator (tut6.html) gained a libration- zoo scenario and physics fixes, the worksheet (worksheet6.data.js) gained a Part E capstone + exam questions, and docs were updated (CHANGELOG, README, new TODO.md). **server.js has a zero diff** — no change to accounts, sessions, roles, prereq gating, input handling, or DoS guards. The new simulator code is self-contained, Three.js from the SRI-pinned CDN, no network calls, no untrusted- data DOM sinks (all worksheet content is trusted-author HTML through the already-audited engine). Re- ran the full suite on fresh isolated servers — **34 functional + 22 security checks + DoS guard, all passing**. academy_data.json confirmed gitignored. Cleared to ship v2.5.

v2.4 — audit result: PASS

The v2.4 change set is **Module 6** (three new client files — tut6.html, worksheet6.html, worksheet6.data.js), Module 4/5 worksheet content edits, and **one small server.js change**: an 'error' handler that exits cleanly on EADDRINUSE (port already in use). That handler touches no request path, auth, session, role, or input logic — the entire auth/gating/DoS surface is unchanged. Module 6 is static client-side files rendered by the already-audited worksheet engine through the same trusted-author HTML path used since v1.0; its simulator (tut6.html) is self-contained with Three.js from the SRI-pinned CDN, no network calls, no untrusted-data DOM sinks. t6 was added to COURSE (server-side prereq chain) — exercised by the suite (course length now 8, prereq gating green). Re-ran the full suite on fresh isolated servers — **34 functional + 22 security checks + DoS guard, all passing**. academy_data.json confirmed gitignored. Cleared to ship v2.4.

v2.3 — audit result: PASS (re-run)

The v2.3 change set is **four worksheet data files** (worksheet2/3a/4/5.data.js) only — enriched author-authored exercise content (teach/predict/think/do prose), rendered by the already- audited engine through the same trusted-author HTML path used since v1.0. **server.js has a zero diff**, so the entire auth/gating/DoS surface is unchanged. No new files, endpoints, inputs, user or network data, or DOM sinks fed by untrusted data. Re-ran the full suite — **34 functional + 22 security checks + DoS guard, all passing**. academy_data.json confirmed gitignored. Cleared to ship v2.3.

v2.2 — audit result: PASS (re-run)

Client-side only; **server.js has a zero diff** from v2.1, so the audited auth/gating/DoS surface is unchanged and re-verified by the suite. Reviewed the touched files: - **resources.html** — now fetches the same-origin worksheetN.data.js files and runs each via Function(text) to read its resources array. This evaluates **same-origin, author-authored** code — the same trust boundary as the <script src> it replaced — with no user input and no cross-origin fetch. (An attacker who could alter those files could already alter the site.) - **worksheet-engine.js** — added rendering of the new teach/predict/think/do fields, all author-authored worksheet content rendered as HTML by design (unchanged trusted-author model). The only dynamic value, ME.name/ME.role, is still esc()-escaped before DOM insertion. - **worksheet-engine.css / worksheet1.data.js** — styling and author content only. No new endpoints, secrets, external calls, or user-input sinks. Re-ran the full suite — **34 functional + 22 security checks + DoS guard, all passing**. academy_data.json confirmed gitignored. Cleared to ship v2.2.

v2.1 — audit result: PASS (re-run)

v2.1 is client-side only; **server.js has a zero diff** from v2.0, so the audited auth/gating/DoS surface is unchanged and re-verified by the suite. Reviewed the touched files: - **tut1.html** — physics/rendering fixes (hyperbolic Kepler propagation, frustum-culling flags, removed the velocity-arrow checkbox). No new inputs, network calls, or injection of untrusted data into the DOM. - **gallery.html** — a new static page of same-origin links (worksheetN.html, tutN.html) with no user input and no innerHTML of external data. - **worksheet-engine.js / resources.html** — the only change was replacing absolute paths ('/', href="/") with relative ones (index.html); no security impact. - Worksheet content (worksheet1.data.js) is author-authored, rendered through the existing esc()-based engine (user/server data still escaped). No new endpoints, secrets, or external calls. Re-ran the full suite — **34 functional + 22 security checks + DoS guard, all passing**. academy_data.json confirmed gitignored. Cleared to ship v2.1.

v2.0 — audit result: PASS (re-run)

The v2.0 worksheet overhaul is entirely **client-side**; server.js has a **zero diff** from v1.3, so the audited auth surface (path traversal, sessions, roles/priv-esc, prerequisite gating, DoS body cap, input validation, timing-safe password compare) is unchanged and re-verified by the suite. Reviewed the new client code: - **XSS**: the only non-author data rendered is ME.name/ME.role (server-held, set by the user at registration) — both are esc()-escaped before innerHTML in worksheet-engine.js. Quiz question/option/feedback text is also esc()-escaped (defense in depth). Author-written worksheet content (objectives/tutorial/inline SVG/exercises) is intentionally rendered as HTML, the same trusted-author model used since v1.0. - **Standalone/localStorage**: stores only booleans keyed by author-defined task IDs, read back via JSON.parse in a try/catch, and used only as truthy completion flags — never re-inserted into the DOM. No injection path, no secrets stored. Note that standalone mode has **no server-side gating** by design (it's for open/self-study); tracked cohorts must run the server. - **resources.html**: loads same-origin, author-authored worksheetN.data.js as <script>; no user input is involved. - **cheatsheet.html / instructor guide**: static content, no inputs. No new endpoints, se-

crets, or external calls (Three.js remains the same SRI-pinned CDN build). Re-ran the full suite — **34 functional + 22 security checks + DoS guard, all passing** (bash test/run.sh). academy_data.json confirmed gitignored. Cleared to ship v2.0.

v1.3 — audit result: PASS (re-run)

Module 5 and the Module 2 rebalance added **no new server attack surface**: everything new is static client-side files (tut5.html, worksheet5.*, and the inert worksheet6.data.js seed), rendered in the browser through the existing esc()-based renderer and using only the already-audited /api/task and /api/complete endpoints. The course expansion to 8 modules (t6/t7/t8 added) is a COURSE-array change only — unlocked() gating logic is unchanged and still walks the prereq chain. Three.js remains the same SRI-pinned CDN build; the new tool loads the moon texture from the same pinned cdn.jsdelivr.net/gh/mrdoob/three.js@r128 path. No new endpoints, secrets, or external calls; new worksheet text is static (no user input interpolated into HTML). Re-ran the full suite — **34 functional + 22 security checks + DoS guard, all passing** (bash test/run.sh). Confirmed academy_data.json remains gitignored. Cleared to ship v1.3.

v1.2 — audit result: PASS (re-run)

Modules 3 and 4 added **no new server attack surface**: both are static client-side files (tut3a.html, tut4.html, and their worksheets/data), rendered in the browser and using only the existing, already-audited /api/task and /api/complete endpoints (prerequisite gating and score clamping enforced server-side). The course re-order (Maneuvers → Module 4; xGEO → 5; Observability → 6) is a COURSE-array change only — the gating logic (unlocked()) is unchanged and still walks the prereq chain. Reviewed: new worksheet data is static text rendered through the existing esc()-based renderer (no new HTML injected from user input); Three.js remains the same SRI-pinned CDN build; no new endpoints, secrets, or external calls. Updated the stale /complete prereq-bypass test (it referenced the removed t3b id) to target t4; re-ran the full suite — **28 functional + 22 security checks + DoS guard, all passing** (bash test/run.sh). Cleared to ship v1.2.

v1.1 — audit result: PASS (re-run)

Module 2 added **no new server attack surface**: it's all client-side rendering plus the existing, already-audited /api/task endpoint (which enforces prerequisites and score clamping server-side). Re-ran the full suite — **25 functional + 22 security checks + DoS guard, all passing** (bash test/run.sh). Durability changes reviewed: atomic temp-file writes + rolling .bak (no new external input; the .bak/.tmp files are gitignored alongside the data file). The optional bring-your-own Claude key remains browser-only (never sent to this server). No new secrets, no new endpoints, no XSS sinks introduced (new user-facing text is static/escaped). Cleared to ship v1.1.

Threat model

- Small-cohort training tool, run on a trusted machine/LAN or a modestly-hosted instance.
- Holds: user names/emails, **script password hashes**, per-user progress. No payment data, no PII beyond name/email.

- First registered account becomes the **instructor** (sees the roster); everyone else is a student.
- Store: single JSON file (academy_data.json) — **gitignored, never committed.**

Audit checklist (run every version)

Area	Check	Method
Path traversal	can any request read files outside public/?	live requests + canary files above web root
AuthN	are protected endpoints unreachable without a session?	unauth requests to /me,/roster,/complete,/task
AuthZ / priv-esc	can a student reach instructor-only data or self-assign a role?	student hits /roster; register with role field
Session integrity	forged/garbage cookies rejected; logout invalidates	crafted sid; logout then reuse
Prerequisite bypass	can the client complete a locked module?	student POSTs locked tutorial to /complete & /task
Input validation	score clamped; weak pw / dup email / bad email rejected	boundary POSTs
DoS	oversized request body doesn't crash the server	300 KB body, then health check
Client XSS	user data escaped before hitting the DOM	grep innerHTML sinks + esc() coverage
Secrets	no keys/tokens committed; data file gitignored	repo-wide grep
Supply chain	external scripts integrity-pinned	Three.js SRI (in tut1.html)

v1.0 — audit result: PASS

Verified safe (all tested live): - **Path traversal — contained.** path.normalize + a startsWith(ROOT + sep) guard reject every ../, encoded, and absolute variant (all 404). Canary files placed above public/ were never served. - **Authentication enforced.** /api/me (401), /api/roster (403), /api/complete (401), /api/task (401) all reject requests with no session. - **No privilege escalation.** A student is denied /api/roster (403). Sending role:"instructor" in the register body is ignored — role is assigned server-side (first user only = instructor). - **Session integrity.** Forged/garbage sid cookies → 401. Logout deletes the session server-side; reusing the old cookie → 401. - **Prerequisite gating is server-side.** A student cannot /complete or /task a locked module (403) — the client cannot unlock ahead by tampering. - **Input validation.** Scores clamped to 0-100; weak password (<8), duplicate email, and malformed email all rejected. Passwords hashed with **scrypt** + per-user random salt; compared with timingSafeEqual (constant-time). - **DoS guard.** Request bodies over 100 KB are dropped (req.destroy()); the

server stays alive and keeps serving normal requests (verified with a 300 KB body). - **No client XSS.** Every innerHTML write of user-controlled data (ME.name) is wrapped in `esc()`. No `eval / new Function(userdata) / document.write`. - **No secrets committed.** `academy_data.json` (password hashes) is **gitignored**. The only “key” in the client is the optional bring-your-own Anthropic key for the AI tutor, held solely in the user’s browser tab and never transmitted to this server or stored.

Accepted (documented, consistent with the trusted-cohort threat model): - **No HTTPS / rate-limiting / CSRF token in this build.** Session cookies are `HttpOnly + SameSite=Strict` (mitigates CSRF and JS cookie theft), but there is no TLS or per-IP throttling. *For any internet-facing deployment: put it behind an HTTPS reverse proxy, add rate-limiting on `/api/login` and `/api/register`, and consider account-lockout on repeated failures.* - **JSON-file store** is fine for a cohort; not concurrent-write-safe at scale. Move to a real DB if usage grows.

Net: no confidentiality/integrity/priv-esc/code-exec issues for the intended use. Cleared to ship v1.0.